Building with AI — A Lean Workflow

A one-page version of the context framework for anyone using AI builders like Replit, Lovable, or Base44.

Most people jump straight to “build me an app.” The AI then guesses everything about who it’s for, what it should feel like, and what it shouldn’t do — and the guesses are usually wrong. These three rules fix that.

Rule 1 — Set the context BEFORE asking for features

Your **very first message** in any new project should not be “build me…”. It should be this:

```
I'm building an app for [WHO will use it].
It solves this problem: [WHAT it does for them, in one sentence].
It should feel: [pick 2–3 words: friendly, playful, calm, bold,
minimal…].
Colors / style: [2–3 colors, or "you pick something that fits"].
Don't add: [what you DON'T want — logins? ads? long forms?].
If anything is unclear, ask before building.
```

Send that. Wait for the AI to acknowledge. Then ask for the first feature.

> Why this works: you only have to say it once, and every feature you ask for after that inherits it. Without it, every prompt is a fresh coin flip.

Bonus — Think on paper first, then photograph it

Before you prompt anything, do the real mental work by hand. Sketch screens. Draw arrows between them. Stick post-its on a wall. Scribble the user journey on a napkin. Then take a photo and paste it into the chat alongside your first message.

Why this matters:

- The thinking happens when you draw, not when you prompt.
- A messy paper wireframe carries more intent than a paragraph of description.

- AI builders are excellent at turning a photo of a sketch into a real screen, flow diagram, or layout.

Use it for:

- **Screen mockups** — boxes, labels, arrows on paper
- **User flows** — post-its: "tap here → see this → go there"
- **Information hierarchy** — what's big, what's small, what's grouped
- **Data shape** — rough sketches of what an item or list contains

The order is the point: **paper → photo → context → prompt**. Don't reverse it.

Rule 2 — When it's wrong, fix the rules, not the output

It's tempting to keep arguing with the AI about the same screen. "Make the button bigger." "No, bigger." "No, more rounded." That's a trap — you'll do it again on the next screen.

When something is wrong, stop and ask: **what rule was I missing?**

Then add it to your context. Examples:

- "All buttons should be at least 48px tall."
- "Never use red — it looks like an error."
- "Always show prices with the currency symbol."
- "When something succeeds, say it in plain language, not 'Success.'"

Send the new rule, then regenerate. Every future output follows it for free.

> The output is the symptom. The rules are the cause. Fix the cause.

Rule 3 — Keep a tiny "lessons learned" note

Make a `LESSONS.md` file in your project. Every time you discover a rule you wish you'd had at the start, write one line.

Examples:

- "Always tell the AI the target user's context."
- "Ask for the mobile view first — it's the harder constraint."
- "If you say 'make it cool,' you get something random. Give 2 specific words."
- "Tell the AI what NOT to add. Otherwise it adds everything."

Next project, paste your lessons into Rule 1 from day one. Your context gets smarter, the AI gets sharper, you stop repeating the same fights.

The whole loop in one sentence

Set the context → ask for the feature → if it's wrong, fix the rules not the output → write down what you learned.

That's it.

Relationship to the full framework

This is the lean variant of the full Context-Driven framework (see the `full-framework` branch). The mapping:

Full framework	Lean version
`/context/`	The first message (Rule 1)
`/specs/`	Rules you add as you go (Rule 2)
`/input/`	Anything you paste in or upload
`/output/`	What the tool produces
`/iterations/`	`LESSONS.md` (Rule 3)

Same philosophy, no folder overhead. Graduate to the full layout when one page stops being enough.